Unit 4: Operators and Expressions in C
Course: C Programming
Program: Bachelor of Information Technology (BIT)
Unit: 4 — Operators and Expressions
Teaching Hours: 4 Hours

1. Learning Objectives

After completing this unit, students should be able to:

1. Define operators and expressions in C.
2. Use arithmetic, relational, logical and assignment operators.
3. Explain type conversion during assignment and expression evaluation.
4. Use increment/decrement and conditional operators.
5. Apply bitwise operators to integer data.
6. Explain sizeof and comma operators.
7. Evaluate expressions using **operator precedence and associativity**.
8. Trace and predict the output of C programs involving different operators.

---

2. Introduction to Operators

Definition

An **operator** is a symbol that tells the compiler to perform a specific operation on one or more operands.
An **operand** is a value or variable on which an operator operates.
For example:
c = a + b;
Here:

- + → operator
- a and b → operands
- = → assignment operator
- c = a + b → expression

Types of Operators in C

Operator Category	Examples
Arithmetic	+, -, *, /, %
Relational	<, >, <=, >=, ==, !=
Logical	&&, `
Assignment	=, +=, -=, *=, /=, %=
Increment/Decrement	++, --
Conditional/Ternary	?:
Bitwise	&, `
Other	sizeof, comma ,

---

3. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Remainder/Modulus	a % b

Example

```
#include <stdio.h>
int main()
{
   int a = 15, b = 4;
   printf("Addition = %d\n", a + b);
   printf("Subtraction = %d\n", a - b);
   printf("Multiplication = %d\n", a * b);
   printf("Division = %d\n", a / b);
   printf("Remainder = %d\n", a % b);
   return 0;
}
```

Output

Addition = 19
Subtraction = 11
Multiplication = 60
Division = 3
Remainder = 3

Important: Integer Division

If both operands are integers:

```
int a = 5, b = 2;
printf("%d", a / b);
```

Output:
2
The fractional part is discarded.
But:

```
printf("%f", (float)a / b);
```

produces:
2.500000

---

4. Relational Operators

Relational operators compare two values.
The result is generally:

- 1 → True
- 0 → False

perator	Meaning
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
!=	Not equal to

Example

```
#include <stdio.h>

int main()
{
   int a = 10, b = 20;

   printf("%d\n", a < b);
   printf("%d\n", a > b);
   printf("%d\n", a == b);
   printf("%d\n", a != b);

   return 0;
}
```

Output

1
0
0
1

Important Exam Point

Do not confuse:
=
with
==
= means **assignment**.
== means **comparison/equality**.
For example:
x = 10;
assigns 10 to x.
x == 10
checks whether x is equal to 10.

---

5. Logical Operators

Logical operators are mainly used to combine conditions.

Operator	Meaning
&&	Logical AND
`	
!	Logical NOT

5.1 Logical AND &&

The result is true only when **both conditions are true**.

A	B	A && B
0	0	0
0	1	0
1	0	0
1	1	1

Example

```
#include <stdio.h>
int main()
{
   int age = 25;
   int citizen = 1;

   if (age >= 18 && citizen == 1)
      printf("Eligible to vote");

   return 0;
}
```

5.2 Logical OR ||

Returns true if **at least one condition is true**.

```
if (marks >= 40 || attendance >= 75)
   printf("Condition satisfied");
```

5.3 Logical NOT !

It reverses the logical value.

```
int a = 10;
printf("%d", !a);
```

Output:

0

Because a non-zero value is considered true, and !true becomes false.

---

6. Assignment Operators

The assignment operator assigns a value to a variable.

Simple assignment

```
int x;
x = 10;
```

Compound Assignment Operators

Operator	Equivalent
+=	x = x + 5
-=	x = x - 5
*=	x = x * 5
/=	x = x / 5
%=	x = x % 5

Example

```
#include <stdio.h>
int main()
{           int x = 10;
                  x += 5;
                        printf("%d\n", x);
                              x *= 2;
                  printf("%d\n", x);
                        x -= 4;
                        printf("%d\n", x);
   return 0; }
```

Output:
15
30
26

---

7. Type Conversion in Assignment
Type conversion occurs when a value of one data type is assigned to a variable of another data type.
Example 1: Integer to float

```
int x = 10;
        float y;
                y = x;
```

Here:
int → float
The value becomes:
10.000000
This is generally a safe conversion.

Example 2: Float to Integer

```
float x = 10.75;
int y;
y = x;
printf("%d", y);
```

Output:
10
The fractional part is discarded.
Important
float → int
may cause **loss of fractional information**.

Example 3

```
#include <stdio.h>
int main()
{
   float price = 99.99;
   int amount;
   amount = price;
   printf("Amount = %d", amount);
   return 0;
}
```

Output:
Amount = 99

---

8. Increment Operator ++
The increment operator increases a value by 1.
x++;
is equivalent to: x = x + 1;
There are two forms:
Post-increment
x++
Pre-increment
++x

Post-increment
The current value is used first, then incremented.

```
int x = 5;
printf("%d\n", x++);
printf("%d\n", x);
```

Output:
5
6

Pre-increment
The value is incremented first, then used.

```
int x = 5;
printf("%d\n", ++x);
```

Output:
6

---

9. Decrement Operator --
The decrement operator decreases a value by 1.

```
x--;
```

is equivalent to:

```
x = x - 1;
```

Post-decrement

```
x--
```

Pre-decrement

```
--x
```

Example:

```
#include <stdio.h>

int main()
{
   int x = 5;
   printf("%d\n", x--);
   printf("%d\n", x);
   return 0;
}
```

Output:
5
4

---

10. Pre-Increment vs Post-Increment
This is a very important examination topic.
Consider:

```
int x = 5;
int y;

y = x++;
```

After execution:
y = 5
x = 6
But:

```
int x = 5;
int y;
y = ++x;
```
After execution:
y = 6
x = 6

Remember
Post: Use first → increment later.
Pre: Increment first → use later.

---

11. Ternary / Conditional Operator
The ternary operator is the only operator in C that requires **three operands**.
Syntax
condition ? expression1 : expression2;
If the condition is true, expression1 is executed.
Otherwise, expression2 is executed.
Example

```
int a = 10, b = 20;
int max;
max = (a > b) ? a : b;
printf("Maximum = %d", max);
```
Output:
Maximum = 20

Example: Even or Odd

```
#include <stdio.h>
int main()
{
   int n;
   printf("Enter a number: ");
   scanf("%d", &n);
   (n % 2 == 0) ? printf("Even") : printf("Odd");
   return 0;
}
```

---

12. Bitwise Operators
Bitwise operators work directly on the **individual bits** of integer values.

Operator	Meaning
&	Bitwise AND
`	`
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift

12.1 Bitwise AND &
Example:
5 = 0101
3 = 0011

& = 0001
Therefore:
5 & 3
is:
1
Program

```
#include <stdio.h>

int main()
{
   int a = 5, b = 3;

   printf("%d", a & b);

   return 0;
}
```

Output:
1

---

13. Bitwise OR |
5 = 0101
3 = 0011

| = 0111
0111 is decimal 7.
printf("%d", 5 | 3);
Output:
7

---

14. Bitwise XOR ^
XOR gives 1 when the two corresponding bits are different.
5 = 0101
3 = 0011
---
^ = 0110
Therefore:
5 ^ 3
gives:
6
XOR Truth Table

A	B	A ^ B
0	0	0
0	1	1
1	0	1

A	B	A ^ B
1	1	0

---

15. Bitwise NOT ~

The ~ operator changes every bit:

0 → 1

1 → 0

Example:

```
int x = 5;

printf("%d", ~x);
```

For signed integers, the exact decimal result depends on the integer representation, but on the common two's-complement systems used today:

~5 = -6

Students should understand that ~ operates on the **binary representation**, not simply on the decimal digits.

---

16. Left Shift <<

Left shift moves bits toward the left.

5 << 1

Binary:

5 = 0101

After one left shift:

1010

which is:

10

Thus:

```
printf("%d", 5 << 1);
```

Output:

10

---

17. Right Shift >>

Right shift moves bits toward the right.

20 >> 2

Binary:

20 = 10100

After two right shifts:

00101

which is:

5

Therefore:

```
printf("%d", 20 >> 2);
```

Output:

5

Practical rule

For positive integers:

x << 1 ≈ x × 2

x >> 1 ≈ x / 2

But students should remember that these are bit operations, not general replacements for multiplication and division in every situation.

18. Other Operators

18.1 sizeof Operator

The sizeof operator returns the size of a data type or variable in **bytes**.

Syntax

sizeof(variable)

or

sizeof(data_type)

Example

```
#include <stdio.h>

int main()
{
   int a;
   float b;
   char c;
   double d;

   printf("int = %zu bytes\n", sizeof(a));
   printf("float = %zu bytes\n", sizeof(b));
   printf("char = %zu byte\n", sizeof(c));
   printf("double = %zu bytes\n", sizeof(d));

   return 0;
}
```

The exact sizes are implementation-dependent.

Important

Do not assume that every C compiler has identical sizes for all data types.

---

19. Comma Operator ,

The comma operator allows multiple expressions to be evaluated from left to right.

Example:

```
int a, b;

a = 10, b = 20;
```

Both assignments are performed.

Example

```
#include <stdio.h>

int main()
{
   int a, b, c;

   c = (a = 10, b = 20, a + b);

   printf("a = %d\n", a);
   printf("b = %d\n", b);
   printf("c = %d\n", c);

   return 0;
}
```

Output:
a = 10
b = 20
c = 30

Important distinction

The comma used to separate function arguments is not necessarily the comma operator.
For example:
printf("%d %d", a, b);
Here commas separate function arguments.

---

20. Expressions in C

Definition

An **expression** is a combination of constants, variables, operators and function calls that produces a value.
Examples:
a + b
x * y + 10
a > b
(x > 0 && y > 0)
(a > b) ? a : b

---

21. Expression Evaluation

Consider:
int x;

x = 10 + 5 * 2;
A beginner might calculate:
10 + 5 = 15
15 × 2 = 30
But C follows **operator precedence**.
Multiplication has higher precedence than addition.
Therefore:
10 + (5 × 2)
= 10 + 10
= 20
So:
x = 20;

---

22. Operator Precedence

Operator precedence determines **which operator is evaluated first** when an expression contains multiple operators.

Important Precedence Table

From higher to lower precedence:

Priority	Operators	Associativity
1	()	Left to right
2	++, --, unary !, ~, unary +, unary -, sizeof	Right to left
3	*, /, %	Left to right
4	+, -	Left to right
5	<, <=, >, >=	Left to right
6	==, !=	Left to right
7	&	Left to right

8	^	Left to right
9	`	`
10	&&	Left to right
11	`	
12	?:	Right to left
13	Assignment operators	Right to left
14	,	Left to right

Important Teaching Point
Students should not try to memorize every precedence level initially. For normal programming, **parentheses are the safest way to make the intended order explicit**.

---

23. Associativity
When two or more operators have the **same precedence**, associativity determines the direction of evaluation/grouping.
Most arithmetic operators use **left-to-right associativity**.
Example:
20 / 5 * 2
Both / and * have the same precedence.
Therefore:
(20 / 5) * 2
= 4 * 2
= 8
Not:
20 / (5 * 2)
= 2

Assignment Associativity
Assignment operators are evaluated/grouped from **right to left**.
Example:
a = b = c = 10;
This is grouped as:
a = (b = (c = 10));
Therefore:
a = 10
b = 10
c = 10

---

24. Worked Expression Examples
Example 1
int x;

x = 10 + 5 * 2;
Evaluation:
5 * 2 = 10
10 + 10 = 20
Therefore:
x = 20

Example 2
int x;

x = (10 + 5) * 2;
Evaluation:
10 + 5 = 15
15 * 2 = 30
Therefore:
x = 30

Example 3
int x = 10;

x += 5 * 2;
First:
5 * 2 = 10
Then:
x += 10
which means:
x = x + 10
Therefore:
x = 20

---

25. Combined Example Program

This program demonstrates several Unit 4 operators.

```
#include <stdio.h>

int main()
{
   int a = 10, b = 3;
   int sum, max;

   /* Arithmetic */
   sum = a + b;

   printf("Sum = %d\n", sum);
   printf("Difference = %d\n", a - b);
   printf("Product = %d\n", a * b);
   printf("Quotient = %d\n", a / b);
   printf("Remainder = %d\n", a % b);

   /* Relational */
   printf("a > b = %d\n", a > b);

   /* Logical */
   printf("(a > 5 && b < 5) = %d\n",
         a > 5 && b < 5);

   /* Ternary */
   max = (a > b) ? a : b;
   printf("Maximum = %d\n", max);

   /* Bitwise */
   printf("a & b = %d\n", a & b);
   printf("a | b = %d\n", a | b);
   printf("a ^ b = %d\n", a ^ b);

   /* Increment */
   a++;
   printf("After increment = %d\n", a);

   return 0;
}
```

26. Common Mistakes by Students

Mistake 1: Using = instead of ==

Wrong:
if (a = 10)
Usually intended:
if (a == 10)

Mistake 2: Confusing / and %

10 / 3 = 3
while:
10 % 3 = 1
/ gives the quotient for integer operands.
% gives the remainder.

Mistake 3: Confusing ++x and x++

y = ++x;
increments first.
y = x++;
uses the old value first.

Mistake 4: Ignoring integer division

float x = 5 / 2;
The result is:
2.0
not:
2.5
because 5 and 2 are integer operands.
Use:
float x = 5.0 / 2;
or:
float x = (float)5 / 2;
to obtain 2.5.

Mistake 5: Ignoring precedence

Instead of relying on memory:
result = a + b * c;
make the intention explicit when appropriate:
result = a + (b * c);
or:
result = (a + b) * c;
depending on the desired calculation.

---

27. Quick Revision Table

Topic	Key Point	Example
Arithmetic	Mathematical operations	a+b
Relational	Compares values	a>b
Logical	Combines conditions	a>5 && b<10
Assignment	Assigns/updates values	x += 5
Type conversion	Converts one data type to another	int x = 3.5;
Increment	Adds 1	x++
Decrement	Subtracts 1	x--
Ternary	Short conditional expression	a>b ? a:b
Bitwise AND	Bit-level AND	a & b
Bitwise OR	Bit-level OR	a | b
Bitwise XOR	Bit-level exclusive OR	a ^ b
Bitwise NOT	Reverses bits	~a
Left shift	Shifts bits left	a << 1
Right shift	Shifts bits right	a >> 1
sizeof	Gives size in bytes	sizeof(int)
Comma	Evaluates expressions left-to-right	(a=1,b=2)
Precedence	Determines operator priority	a+b*c
Associativity	Determines grouping when precedence is equal	a/b*c

---

28. Practice Questions for BIT Students

A. Short Questions

1. What is an operator?
2. What is an operand?
3. What is an expression?
4. Differentiate between = and ==.
5. What is the difference between / and %?
6. What is type conversion?
7. Differentiate between pre-increment and post-increment.
8. What is a ternary operator?
9. What is the purpose of the sizeof operator?
10. What is operator precedence?
11. What is associativity?
12. What are bitwise operators?

B. Predict the Output

Question 1

```
int a = 5, b = 2;

printf("%d", a + b * 3);
```

Answer:

11

Question 2

```
int x = 5;

printf("%d ", x++);
printf("%d", x);
```

Answer:

5 6

Question 3
int x = 5;

printf("%d ", ++x);
printf("%d", x);
Answer:
6 6

Question 4
int a = 10, b = 20;

printf("%d", a > b);
Answer:
0

Question 5
printf("%d", 5 & 3);
Answer:
1

---

29. Practical Programming Exercises
Students should implement the following programs in the laboratory:

1. **Arithmetic Calculator**
   Input two numbers and display addition, subtraction, multiplication, division and remainder.
2. **Largest of Two Numbers**
   Use the ternary operator.
3. **Even or Odd**
   Use the modulus operator.
4. **Positive, Negative or Zero**
   Use relational and logical operators.
5. **Pre/Post Increment Demonstration**
   Write a program demonstrating the difference between ++x and x++.
6. **Bitwise Calculator**
   Input two integers and display:
7. AND
8. OR , XOR
9. **Expression Evaluation**
   Write a program to demonstrate operator precedence and associativity.
10. **Data Type Size Program**
    Use sizeof() to display the size of:
11. char
12. int
13. float, double

⭐ **Key Concept for Students**
When solving any C expression, teach yourself to follow this sequence:
1. Parentheses → 2. Unary operators → 3. * / % → 4. + - → 5. Relational → 6. Equality → 7. Logical → 8. Conditional → 9. Assignment → 10. Comma